

scheduler_eval

A Scheduling Agent with Multi-Level Evaluation Harness

Date: 2026-09-29

Author: Srihar

Repo: github.com/Srihari4589/scheduler-eval

1. Project Overview

scheduler_eval is a weekly meeting-scheduling agent built to practice evaluation. The agent receives a list of meetings with hard constraints (no double-booking, fixed days, earliest-start times, blocked times) and soft preferences (e.g. 'ideally before 11am'), and must place them into a Mon-Fri, 09:00-17:00 week. Some scenarios include a mid-task correction that forces the agent to revise its schedule without breaking earlier constraints.

The project follows Hamel Husain's three-level evaluation framework: Level 1 (deterministic unit tests), Level 2 (LLM-based qualitative judgment), and Level 3 (A/B testing). The correction step is the key design feature: a one-shot agent cannot produce retries, corrections, or 'did it forget what I told it earlier' failures.

2. Architecture

The project is organized into the following components:

- schemas.py - Pydantic data models (Meeting, ScenarioInput, ScheduleOutput, Correction)
- agent.py - Provider protocol + GeminiProvider (real model) + FixtureProvider (naive baseline)
- grader.py - Deterministic scoring: hard violations, preference score, correction compliance, regression check
- judge.py - LLM-based qualitative judge (GeminiJudge) + negative control (StubbornJudge)
- runner.py - End-to-end scenario runner, saves JSON reports to artifacts/runs/
- cli.py - Typer CLI with --provider, --model, --repeats, and --judge flags
- tests/ - 24 tests covering grader, runner, scenarios, and judge
- scripts/ - Review tool, agreement measurement, Loggy import, report generation

3. Test Results

All 24 tests pass. The test suite covers:

- Grader: 9 tests - hard violations, double-booking, missing meetings, work hours, not_before/not_after, blocked times, regression, correction compliance
- Runner: 6 tests - perfect provider, stubborn provider (negative control), correction exercised, turn-1 validity
- Scenarios: 3 tests - loading, uniqueness, correction ground truth
- Judge: 3 tests - stubborn judge pass/fail, dimension coverage

4. Benchmark Results (Gemini 3.5 Flash Lite)

Latest run: 2026-09-29T20:35:28.667624+00:00

Provider: gemini:gemini-3.5-flash-lite

Cases: 6 | Errors: 0

Hard pass rate:	1.0000
Avg preference score:	1.0000
Avg regression score:	1.0000
Avg correction exercised:	1.0000
Avg added meeting present:	1.0000
Avg attendee moved off unavailable day:	1.0000
Avg removed meeting absent:	1.0000
Avg new blocked time respected:	1.0000

5. Test Scenarios (6 total)

- `baseline_week`: No correction. Basic mix of constraints: shared attendees, fixed day, not-before, soft preference.
- `add_meeting_midtask`: Correction adds a new meeting. Tests fitting it in without breaking earlier constraints.
- `attendee_unavailable_midtask`: Sam's meetings pinned to Tuesday. Correction makes Tuesday unavailable - agent must move them.
- `remove_meeting_midtask`: Correction cancels a meeting. Tests clean removal without disturbing anything else.
- `blocked_time_midtask`: Wednesday afternoon blocked. Correction blocks morning - agent must fit meetings in 90-min window.
- `dense_week_add_big_meeting`: 7 meetings, 4 people, daily lunch blocked. Correction adds a 2-hour meeting with a deadline.

6. Key Findings

- False positive found and fixed: `attendee_unavailable_midtask` originally never put anything on Tuesday, so the correction tested nothing. Fixed by forcing Sam's meetings onto Tuesday with `fixed_day`.
- Second grader bug found: regression check still expected a cancelled meeting, so a correct agent scored 0. Fixed in `runner.py`.
- Negative control added: `StubbornProvider` ignores every correction. Found that a perfect score could measure nothing - grader now adds a hard violation when an unavailable attendee is still booked that day.
- Prompt ambiguity fixed: model read 'before 10:00' as 'by 10:00'. Reworded prompt to say 'strictly earlier than... exactly equal does NOT satisfy it'. Preference score went from 0.9444 to 1.0000.
- Rate limit handling: Gemini free-tier 429 errors invalidate --repeats runs. Documented in known issues.

7. Evaluation Levels Implemented

Level 1: Unit Tests (Complete)

24 deterministic tests covering grader, runner, scenarios, and judge. Run on every push via GitHub Actions CI.

Level 2: LLM Judge (Implemented)

GeminiJudge critiques schedules on 5 qualitative dimensions: buffer time, meeting grouping, energy management,

fairness, practicality. StubbornJudge serves as negative control. Meta-evaluation workflow built (review tool + agreement measurement).

Level 3: A/B Testing (Partially)

--repeats N runs the benchmark multiple times for variance analysis. Loggy import script enables run comparison. Full A/B testing (two prompts, same scenarios) not yet automated.

8. Known Issues and Limitations

- Constraint satisfaction does not equal schedule quality - grader verifies hard rules, not goodness
- No soft-preference gradation (binary per meeting, no partial credit)
- No attendee workload fairness check
- No temporal distribution check (all meetings on 2 days = evenly spread)
- --repeats summary silently skips errored cases
- No tracing / observability (no Langfuse integration)
- No human evaluation workflow (tooling built, labels not yet collected)
- No automated A/B testing
- Gemini free-tier rate limits require spacing between runs

9. Comparison with Teammate's transcription_agent_eval

The teammate's project is at a more mature evaluation phase. Key differences:

- Meta-evaluation: Teammate has full workflow (human annotation, evaluator comparison, adjudication). We have the tooling built but need human labels.
- Tracing: Teammate uses Langfuse for every benchmark item. We save JSON reports but have no trace UI.
- CI: Teammate runs tests + fixture benchmark on every push. We now have the same.
- Known defects: Teammate documents as baseline counterexamples. We now document 9 known issues.
- Experiment tracking: Teammate uses Loggy. We have an import script ready.
- Data policy: Teammate has consent/redaction/retention for sensitive transcripts. Our data is synthetic, so N/A.

10. Next Steps

- Collect human labels using the review tool, then measure judge precision/recall
- Add harder scenarios: two corrections in a row, correction conflicting with blocked time, impossible correction
- Try a weaker model to confirm the grader discriminates good from bad
- Integrate Langfuse tracing for browsable runs
- Fix --repeats summary to warn about errors
- Add soft-preference gradation and attendee fairness metrics
- Write the 'what good means' document (half-page definition of schedule quality)
- Rotate Gemini API key (was visible in screen recording)